BSc (Hons) in **Computer Science**

SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 02

> **Objective & Lecture Mapping:** In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

> To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an* `__init__` *method to store history.*
   Example Logic: `IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward`
3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
   Example Logic: `IF wall_ahead AND left_is_visited THEN turn_right`
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

> Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

The ideal table-driven agent would require a full table that associates every possible state of the world, percept, and percept sequence with the appropriate action. The size of such a table would be immense and would increase exponentially with the complexity of the environment, like Chess. With increasing lifetime of the agent, it experiences more and more states; the size of the set of percept sequences grows rapidly, so that the required table becomes impossible to construct or even to store. Essentially, the environment is too complex and too dynamic for a perfect mathematical table to be ever created. This is the very reason why intelligent agents are created as software programs that have rules, memory, and reasoning capabilities.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent`. Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

Condition-Action Rules are represented in the SimpleReflexAgent in the if/elif/else blocks in sense_and_act() in agent.py. If percept.get('food_here') is True, it implies that food is available, and therefore, the agent responds by moving 'Up.' In case the former condition is False while percept.get('wall_ahead') is True implying that there is a wall ahead, then the agent's response will be 'Left.' However, in case none of the above conditions is met, the agent responds with the action indicated in the else block which is 'Right.' The above lines are perfect examples of Condition-Action Rules according to the definition in the lecture because the agent is not anticipating or recalling any past experiences but responding to its current perceptions.

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

This is how the SimpleReflexAgent failed. It was not a "world model" agent that can reason based on all its percept history up until now. The SimpleReflexAgent had only the information about the current local percept and applied just one rule, which was that if there is food, then move toward it; if there is a wall ahead, then turn; otherwise just move forward. As mentioned above, the environment was purposely created as partially observable in this laboratory setting, the agent did not have the perception of the whole grid and where it was exactly located, it just had local perceptions like wall_ahead or food_here. Thus, it had no way of knowing whether it had been into this very local state before and thus was currently trapped in a corner or a U-trap. Also, the agent did not know whether it was currently going to repeat its

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent`. Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

In ModelBasedAgent, state is stored in self.last_action, self.last_percept and self.visited_states of agent.py file. It enables the agent to keep its state representation of environment instead of being a blind reactor.
Transition model is defined by how the state of the environment changes after executing an action, in the above code it is illustrated through updating the memory after each decision; storing the current percept, storing the last executed action, and storing previous decision states. Hence it not only reacts to the current environment but also remembers how its past actions have impacted the local state of the environment.
Sensor model represents the process of how the agent's sensors understand the environment. In the provided code, it is shown through the percepts food_here and wall_ahead. With the

Finalize and Lock Lab 02 Documentation